

```

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAX_STATES 20
#define MAX_SYMBOLS 10

typedef struct {
    int transitions[MAX_STATES][MAX_SYMBOLS][MAX_STATES];
    int epsilon_transitions[MAX_STATES][MAX_STATES];
    int num_states;
    int num_symbols;
} EpsilonNFA;

void initializeEpsilonNFA(EpsilonNFA* enfa, int num_states, int num_symbols) {
    enfa->num_states = num_states;
    enfa->num_symbols = num_symbols;

    for (int i = 0; i < num_states; i++) {
        for (int j = 0; j < num_symbols; j++) {
            for (int k = 0; k < num_states; k++) {
                enfa->transitions[i][j][k] = -1;
            }
        }

        for (int k = 0; k < num_states; k++) {
            enfa->epsilon_transitions[i][k] = -1;
        }
    }
}

void addTransition(EpsilonNFA* enfa, int from_state, int symbol, int to_state) {
    for (int i = 0; i < MAX_STATES; i++) {
        if (enfa->transitions[from_state][symbol][i] == -1) {
            enfa->transitions[from_state][symbol][i] = to_state;
            break;
        }
    }
}

void addEpsilonTransition(EpsilonNFA* enfa, int from_state, int to_state) {
    for (int i = 0; i < MAX_STATES; i++) {
        if (enfa->epsilon_transitions[from_state][i] == -1) {
            enfa->epsilon_transitions[from_state][i] = to_state;
            break;
        }
    }
}

void findEpsilonClosure(EpsilonNFA* enfa, int state, bool visited[], int closure[]) {
    if (visited[state]) {
        return;
    }

    visited[state] = true;
    closure[state] = 1;

    for (int i = 0; i < enfa->num_states; i++) {
        if (enfa->epsilon_transitions[state][i] != -1) {
            findEpsilonClosure(enfa, enfa->epsilon_transitions[state][i],
visited, closure);
        }
    }
}

void convertToNFA(EpsilonNFA* enfa) {

```

```

    int epsilon_closure[MAX_STATES][MAX_STATES] = {0};

    for (int i = 0; i < enfa->num_states; i++) {
        bool visited[MAX_STATES] = {0};
        findEpsilonClosure(enfa, i, visited, epsilon_closure[i]);
    }

    printf("\nEpsilon closures:\n");

    for (int i = 0; i < enfa->num_states; i++) {
        printf("Epsilon closure of state %d: { ", i);
        for (int j = 0; j < enfa->num_states; j++) {
            if (epsilon_closure[i][j]) {
                printf("%d ", j);
            }
        }
        printf("}\n");
    }

    printf("\nNFA transitions:\n");

    for (int state = 0; state < enfa->num_states; state++) {
        for (int symbol = 0; symbol < enfa->num_symbols; symbol++) {
            bool reachable_states[MAX_STATES] = {0};

            for (int i = 0; i < enfa->num_states; i++) {
                if (epsilon_closure[state][i]) {
                    for (int j = 0; j < MAX_STATES; j++) {
                        int target = enfa->transitions[i][symbol]
[j];

                        if (target != -1) {
                            reachable_states[target] = true;
                        }
                    }
                }
            }

            printf("From state %d on symbol %d: { ", state, symbol);

            for (int i = 0; i < enfa->num_states; i++) {
                if (reachable_states[i]) {
                    printf("%d ", i);
                }
            }

            printf("}\n");
        }
    }
}

int main() {
    int num_states, num_symbols;
    EpsilonNFA enfa;

    printf("Enter the number of states: ");
    scanf("%d", &num_states);
    printf("Enter the number of symbols: ");
    scanf("%d", &num_symbols);

    initializeEpsilonNFA(&enfa, num_states, num_symbols);
    int num_transitions, from_state, to_state, symbol;

    printf("Enter the number of transitions: ");
    scanf("%d", &num_transitions);

    printf("Enter transitions (from_state symbol to_state):\n");

```

```
    for (int i = 0; i < num_transitions; i++) {
        scanf("%d %d %d", &from_state, &symbol, &to_state);
        addTransition(&enfa, from_state, symbol, to_state);
    }

    int num_epsilon_transitions;
    printf("Enter the number of epsilon transitions: ");
    scanf("%d", &num_epsilon_transitions);

    printf("Enter epsilon transitions (from_state to_state):\n");

    for (int i = 0; i < num_epsilon_transitions; i++) {
        scanf("%d %d", &from_state, &to_state);
        addEpsilonTransition(&enfa, from_state, to_state);
    }

    convertToNFA(&enfa);

    return 0;
}
```